



亞洲大學
ASIA UNIVERSITY

Midterm Project Report

Advanced Computer Programming

Student Name : Salomo Alden CS

Student ID : 114021148

Teacher : DINH-TRUNG VU

2026-04

Chapter 1 Introduction

1.1 Github

- 1) **Personal Github Account:** <https://github.com/salacs>
- 2) **Group Project Repository:** <https://github.com/wangyalal/Group1>

1.2 Overview

My program is an automated web scraper built with the Scrapy framework to collect repository metadata from GitHub. It navigates through user profile pages and crawls into individual repository links to extract specific project details.

The advanced language features and libraries used include:

- Scrapy: Used for asynchronous crawling and request management.
- CSS & XPath Selectors: Employed to target data within GitHub's complex HTML structure precisely.
- Scrapy Items: Implemented to define a consistent data schema for all extracted records.
- Anti-Bot Measures: Configured custom User-Agents and disabled `ROBOTSTXT_OBEY` to ensure successful access to public data.

Results:

The program effectively extracted the URL, Name, About description, Programming Language, and Last Updated Date for all repositories under the target profiles, saving the final data into a structured XML format.

Implementation

1.1 Class 1: GitHubSpider

This class inherits from scrapy.Spider serves as the primary engine. It defines where the crawl starts and how the incoming HTML should be processed.

1.1.1 Fields

- name: The unique identifier for the spider ("github").
- start_urls: The initial list of GitHub repository tab URLs.

1.1.2 Methods

- parse(self, response): Analyzes the main repository list. It identifies repository links and uses response.follow to navigate to the individual repo pages. It also detects the "Next" button to handle pagination.
- parse_repo(self, response): A specialized callback method that extracts the "deep" data (like language and date) once inside a specific repository's page.

1.2 Class 2: GithubItem

Located in items.py, this class defines the data schema. It acts as a formal template for every repository scraped, preventing errors from missing fields or typos.

1.2.1 Fields

- url – URL of the repository
- name – Repository name
- about – Repository description
- language – Primary programming language used
- last_updated – Date of last update

1.3 Method/Function 1: parse()

This method:

- Processes GitHub repository listing pages
- Selects repository elements using CSS selectors
- Extracts repository URLs and descriptions
- Sends requests to individual repository pages using response.follow
- Passes metadata (such as description) using meta
- Handles pagination to continue scraping all repositories

1.4 Method/Function 2: parse_repo()

This method:

- Processes individual repository pages
- Extracts structured repository information using CSS/XPath
- Creates a GithubItem object
- Stores extracted data fields into the item
- Returns the item to the Scrapy pipeline for output

Chapter 2 Results

1.1 Result 1

The scraper successfully extracts repository URLs and names from multiple GitHub user accounts.

1.2 Result 2

The program collects structured repository data, including:

- Repository name
- Description
- Programming language
- Last update date

This data can be exported into JSON, CSV, or XML formats for further analysis.

Chapter 2 Conclusions

This project demonstrates the use of Scrapy for real-world web data extraction. It successfully automates the process of collecting structured data from GitHub user repositories.

Key takeaways include:

- Understanding how to navigate multi-page web structures using Scrapy
- Using CSS and XPath selectors effectively
- Passing metadata between parsing functions
- Structuring scraped data using custom items

However, GitHub's dynamic page structure can sometimes limit data availability, showing the importance of adapting scraping strategies or using more advanced tools when necessary.